

CRAFTYFY TEST PLAN

Project ID :

Project Name : Craftyfy

URL: <https://craftyfy-frontend.vercel.app/>

Tested By : QA Team

Date : 12-12-2025

1. Introduction

Craftyfy is an e-commerce platform designed to showcase and sell handmade craft products. It enables the users to browse products, add items to cart and place orders. This test plan outlines the overall strategy, objectives, scope, and approach to validate the functionalities of the application.

2. Scope

2.1 In-Scope

Functional Testing

- User Registration and Login
- Viewing product details
- Product search and filtering
- Cart Management
- Manage profile
- Order Placement and Tracking
- Subscription and Careers options

User Acceptance Testing(UAT)

- Check overall user experience and usability
- Verify order tracking flow

2.2 Out-of-Scope

- Admin Product Management
- Inventory Management

- Real Payment Transactions
- Stress and Load testing

3. Objectives

- **Verify Functional Correctness:** Ensure that all core features of the Craftyfy web application (user registration, login, product browsing, cart, checkout, order tracking, and subscription) work as per the specified requirements.
- **Identify and Report Defects:** Detect defects during testing, classify them based on severity and priority, and report them in JIRA to reduce risk before production deployment.
- **Validate User Interface and Usability:** Confirm that UI elements are consistent, user-friendly, responsive, and aligned with the expected design.
- **Verify Integration Between Modules:** Validate smooth interaction between different modules such as login, product catalog, cart, payment flow, and order management.
- **Validate Error Handling and Messages:** Ensure proper validation messages, error prompts, and confirmation messages are displayed for invalid inputs and system actions.
- **Confirm Compatibility Across Environments:** Validate application behavior across different browsers and execution environments (local, Docker, and cloud execution).
- **Automate Critical Test Scenarios:** Automate high-priority test cases using Selenium, TestNG, and Maven to improve test coverage and execution efficiency.
- **Generate Test Evidence and Reports:** Produce execution reports (TestNG reports), screenshots on failure to provide traceability and testing evidence.

4. Test Approach

4.1 Test Strategy

Testing for the Craftyfy web application will be carried out using a hybrid approach, combining manual testing and automation testing to ensure maximum coverage, accuracy, and reliability.

4.2 Manual Testing

- Manual testing is performed to validate the functional, UI, and usability aspects of the Craftyfy web application.
- Test scenarios and test cases are first designed and documented in a Test Case Document based on the application requirements and user stories.
- The test cases are then imported into TestRail for centralized test management.
- TestRail is used to organize test cases into sections and execute test cases and record results (Pass/Fail/Blocked).

4.3 Automation Testing

- Automation is implemented for high-priority test cases.
- Selenium WebDriver with Java is used for browser automation.
- TestNG framework is used for Test execution
- Page Object Model (POM) design pattern is followed to improve maintainability and reusability.

4.4 Test Execution

- Tests are executed in local machine, docker container and cloud execution environment.
- Maven is used as the build and dependency management tool.
- Automated tests are executed via IDE and CI pipeline.

4.5 Defect Management

- Defects identified during testing are logged in JIRA.
- Each defect includes steps to reproduce, expected vs actual results, severity and priority along with screenshots.
- Defects are linked to relevant test cases and user stories where applicable.

5. Test Items

- User Registration and Login module
- Product Lists and Filters
- Cart and Orders
- Newsletter Subscription
- Navigation and UI Elements

6. Test Environment

- **Hardware:** System with minimum 8GB RAM.
- **Software:**
 - TestRail for Test Management
 - Selenium for Test Automation
 - JIRA for defect tracking
 - GitHub for Version Control.
 - Jenkins for Continuous Integration
 - Docker for Containerization
 - AWS for Cloud Execution
- **Browsers:** Mozilla Firefox, Microsoft Edge, Google Chrome.

7. Entry/ Exit Criteria

7.1 Entry Criteria

- Requirements are fully understood and documented.
- Test data is prepared.
- The test environment is ready.
- Test cases are prepared.

7.2 Exit Criteria

- All requirements are satisfied.
- High test coverage is achieved.
- All test cases executed and passed.

- All bugs are detected and fixed.
- No severe bugs remain.

8. Risks and Mitigation

- **Cart data loss after logout**
Impact: Poor user experience
Mitigation: Test cart persistence across sessions
- **Browser Compatibility Issues**
Impact: Features may not work well on all browsers.
Mitigation: Test on multiple browsers
- **Incomplete Test Coverage**
Impact: Defects missed before release
Mitigation: Use Requirement Traceability Matrix
- **Time Constraints**
Impact: Reduced testing quality
Mitigation: Critical user flows must be prioritized.
- **Incomplete Test Data**
Impact: Some user flows may not be fully tested.
Mitigation: Prepare and validate complete test data before testing.

9. Test Schedule

Testing activities will be executed in three sprints. Sprint 1 focuses on test planning, test case preparation, manual test execution, defect logging, and test reporting from **Dec 11 to Dec 17**. Sprint 2 covers automation framework setup, automated test execution, and database validation from **Dec 18 to Dec 23**. Sprint 3 includes CI/CD integration, Docker setup, cloud execution, and final project presentation from **Dec 24 to Dec 30**.

10. Test Deliverables

- Test Plan

- Test Case Document
- Requirement Traceability Matrix(RTM)
- Bug Report
- Test Summary Report
- JIRA Sprint Report
- Selenium with TestNG Automation Frameworks
- Automated test scripts
- GitHub Repository with Source code and Documentation
- Jenkins pipeline configuration
- Dockerfile